

RECOMMENDATION SYSTEMS

Building a “Did You Forget?” Recommendation System

Sanjana Nair -Kaggle Recall@5 = 0.26829 (best submission)

1. Problem and Data

Online grocery customers routinely forget items at checkout. The task: for each customer’s most recent order -of which only 70–80% of SKUs are revealed in *last_orders_subset.csv* -predict 5 SKUs likely to be among the withheld (“forgotten”) items. Historical behaviour is given in *all_except_last_orders.csv*: 28,984 purchase rows, 638 customers, 2,595 orders, 632 SKUs, with columns Order, SKU, Member, Delivery Date. Evaluation is Recall@5: the average fraction of withheld items recovered by the 5 recommendations, over all 638 last orders. Comparing average visible basket size (8.6) with average historical order size (11.2) implies the organizers withheld $\approx 23\%$ of each last order -a fact we used to calibrate our offline simulation.

2. The Decisive Methodological Lesson: Leakage-Free Offline Evaluation

With 30 total Kaggle submissions, reliable offline evaluation is the scarcest asset. Our first harnesses were misleading: a hand-tuned model estimated 0.37 offline but scored 0.258 online, and an early learning-to-rank model estimated 0.59 offline yet scored 0.183 online. The cause in both cases was feature leakage: similarity matrices, popularity counts, and SVD factors were computed on the full history, which contained the very orders used as validation targets -the features had effectively memorized the answers.

The fix mirrors deployment exactly. We simulate the competition inside the training file: each customer’s last (and 2nd/3rd-to-last) orders are held out as targets, 20–30% of their items are hidden with reproducible masks, and **every co-purchase structure is rebuilt excluding the target orders**. After this fix, offline estimates predicted the leaderboard within ± 0.001 (e.g., offline 0.2589 $\rightarrow$ online 0.25797), turning offline evaluation into a trustworthy referee for every subsequent decision. A second discipline proved equally important: **one selection decision per experiment**, made by 3-fold cross-validation over customers -an experiment that tuned several knobs on a single small validation split caused a genuine leaderboard regression (0.24585).

3. Key Diagnostic: Forgotten Items Are Not Mostly Repurchases

The grocery literature (Amazon “Buy It Again”, ReCANet) emphasizes repeat-purchase prediction. Our harness let us measure the premise directly: across $\sim 1,500$ simulated cases, only **$\sim 47\%$ of hidden items had ever been bought before** by that customer -more than half of the “forgotten” items are first-time purchases. This single number shaped the feature set (similarity of candidates to the customer’s entire history; category-level signals), justified learning the weighting between repeat and discovery signals with a model rather than hand-fixing it, and eliminated a restrict-to-repurchases idea before it cost a submission.

4. System Architecture

4.1 Features (48 per customer–candidate pair)

Group	Signals
Repeat-purchase engine	Purchase count and rates, recency-decayed frequency, orders/days since last purchase, purchase-interval mean & std, overdue (“due-ness”) score, consecutive-order streak, presence in the last one/two orders
Current-basket evidence	Association-rule confidence $P(\text{candidate} \mid \text{cart item})$ and item-item cosine summed/maxed over the visible cart; customer affinity from the customer’s own past baskets that overlap today’s cart
Embeddings & latent factors	PPMI-SVD item embeddings (word2vec-equivalent) -similarity to the cart and to the customer’s full history; SVD matrix-factorization score; SVD-40 latent item similarity; user-kNN collaborative filtering
Category level	KMeans pseudo-categories over item embeddings: the customer’s purchase rate in the candidate’s category, orders since the category was last bought, category overdue score, whether the cart already covers it, item’s rank within its category
Global & context	Item popularity, distinct buyers, global reorder rate, $P(\text{buy again} \mid \text{bought once})$, basket size, history length, customer reorder tendency

4.2 Five independent model families, fused by vote

Rank/vote fusion of near-equal models is a production-standard ensemble technique, but it only gains from **genuine disagreement**. We verified this the hard way: five rankings derived from one trained model fused to 0.26084, while five separately trained models fused to 0.26829. The final system therefore trains five families that differ along every available axis -feature subset, hyperparameters, random seeds, ranking recipe, and training masks (separately trained runs):

Family	Features	Model	Ranking recipe
linear_blend	4 signals	Hand-tuned linear blend	CF cosine + date-decay recency + latent similarity + affinity
gbm_compact	31	HistGB, lr 0.10, 15 leaves	Heuristic-GBM rank blend
gbm_extended	42	HistGB, lr 0.06, 31 leaves	Rank blend + linear blend
gbm_deep	48	HistGB, lr 0.04, 63 leaves	Rank blend + linear blend
gbm_broad	48	HistGB, lr 0.06, 31 leaves	Self-vote across its own three rankings

Each family produces a top-5 list per order; a position-weighted vote (weights 1.0, 0.9, 0.8, 0.7, 0.6; average-rank tiebreak) selects the final 5 recommendations. Every GBM is itself a bagged ensemble, each member trained on all positives plus its own random draw of negatives.

5. Results

Approach	Offline (honest)	Kaggle Recall@5
Hand-tuned linear blend	0.37 (leaky estimate)	0.258
GBM learning-to-rank with leaky features	0.59 (leaky estimate)	0.183
Single GBM ranker, leak-free evaluation	0.245–0.254	0.245–0.257
GBM + linear-blend fusion + category features	0.259	0.258–0.259
Vote fusion of 3 independently trained models	—	0.264
Vote fusion of 5 independently trained models (final)	—	0.26829
Control: 5 rankings from one model, same voting	—	0.26084

The best submission achieved **Recall@5 = 0.26829**. The progression shows two step-changes: first from honest evaluation enabling reliable feature/model selection, then from fusing independent models. The control row isolates the independence effect: identical voting over non-independent rankings recovers less than half of the fusion gain.

6. Lessons Learnt

(1) Leakage is the silent killer. Both early offline estimates were fantasies produced by similarity structures that had seen the validation orders. Rebuilding structures per validation fold made offline scores leaderboard-accurate to ± 0.001 -the single highest-value fix of the project.

(2) Diagnose before optimizing. Measuring the 47% repurchase share redirected weeks of feature work toward never-bought candidates and killed a plausible-sounding dead end without spending a submission on it.

(3) One decision per experiment. Selecting several knobs on one 399-case validation split caused a real regression; cross-validated selection of a single champion per run fixed it. Submission budget should be spent on validated hypotheses, not exploration.

(4) Ensemble gains come from independence, not from the voting formula. Diversity across feature sets, hyperparameters, seeds, and training masks -not a cleverer combiner -produced the final +0.01. This is the central design principle of the submitted system.

7. Deliverables

The accompanying notebook (*GR12_MLUL2_RecSys.ipynb*) generates the single-run submission end-to-end (Recall@5 = 0.26084) and then applies the vote-fusion step across five model outputs produced during the campaign, yielding the best submission (0.26829) and writes *GR12_rec_5_sets.csv*. Because model training is stochastic and ensemble components from earlier runs are averaged into our best leaderboard file, a fresh run reproduces the method and lands in the same score neighborhood rather than reproducing the file byte-for-byte -the standard caveat for any ensemble system.

Appendix-Experimental Work and Alternative Approaches

Beyond the final system, the team tested a wide range of alternatives before settling on weighted vote fusion of five independently trained model families. These included several weighted blends of heuristic, GBM, and Team Blend scores (differences proved statistically indistinguishable); raw-score versus rank-based fusion (rank-based was more robust to scale differences between models); explicit seen/unseen quotas constraining how many previously purchased items could appear in the top 5 (helped isolated folds but did not generalize); multiple full candidate submissions fused via different positional-weight schemes; and an automated search over blend coefficients, abandoned once returns plateaued. Ideas tested but excluded from the final model included additional weighted blends, alternative combinations of models for the vote, and further positional-weight tuning -none beat the five-family, weighted-vote design. The consistent finding: ensemble diversity across feature sets, hyperparameters, seeds, and training masks drove essentially all of the gain over any single model, while the choice of combiner formula mattered comparatively little.

Course techniques evaluated but not retained in the final system include: PageRank item importance on the co-purchase graph (tested as a popularity replacement; did not outperform raw popularity), ALS matrix factorization (truncated SVD was chosen as its deterministic equivalent at this data scale), full Apriori itemset mining (pairwise support–confidence–lift statistics sufficed for candidate scoring), and content-based filtering (inapplicable, as no product name or category metadata was provided). The ~53% of forgotten items that are first-time purchases is a cold-start sub-problem, addressed through history-similarity embeddings and category-level features rather than item metadata.